

MAGE-Scrum

Multi-Agent Generative Ecosystem for Automated Scrum Development

PROJEKTTEAM

Fabian Rahofer
Hannes Zelger

TECHNOLOGIEN

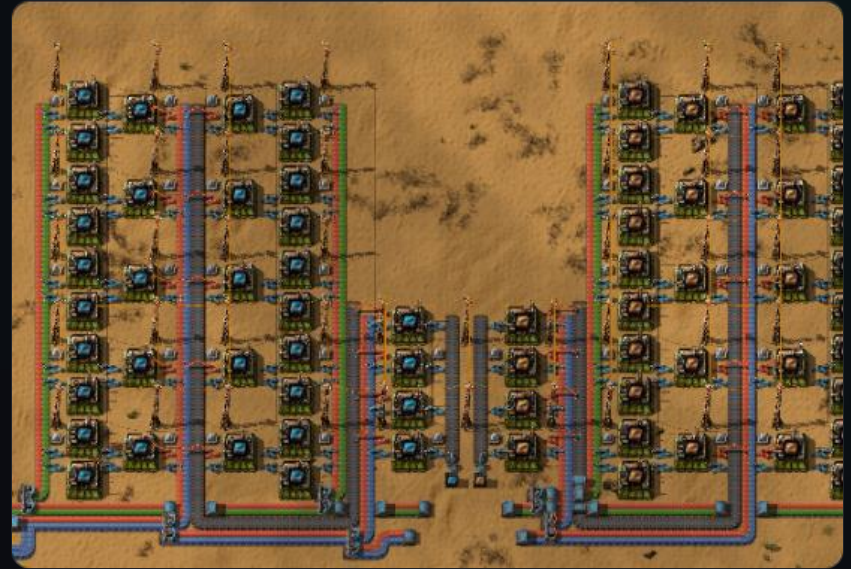
CrewAI • LangGraph • Ollama • .NET 8

PROJEKTZIEL & VISION

Entwicklung eines **autonomen Multi-Agenten-Systems**, das ein vollständiges Scrum-Team simuliert:

- Analyse von komplexen Anforderungen
- Iterative Softwareentwicklung
- Automatisierte Testgenerierung & Fehlerbehebung
- Erzeugung von auditierbaren Releases

Zielanwendung: Factorio Modeler (Produktionsplaner)



| KERNIDEE: ROLLENISOLIERUNG



Product Owner

Anforderungen &
Gherkin Stories.
Definiert das "Was".



Scrum Master

Sprintplanung &
Auditing. Prozess-
Gatekeeper.



Developer

C# Implementierung &
Dokumentation. Baut
das System.

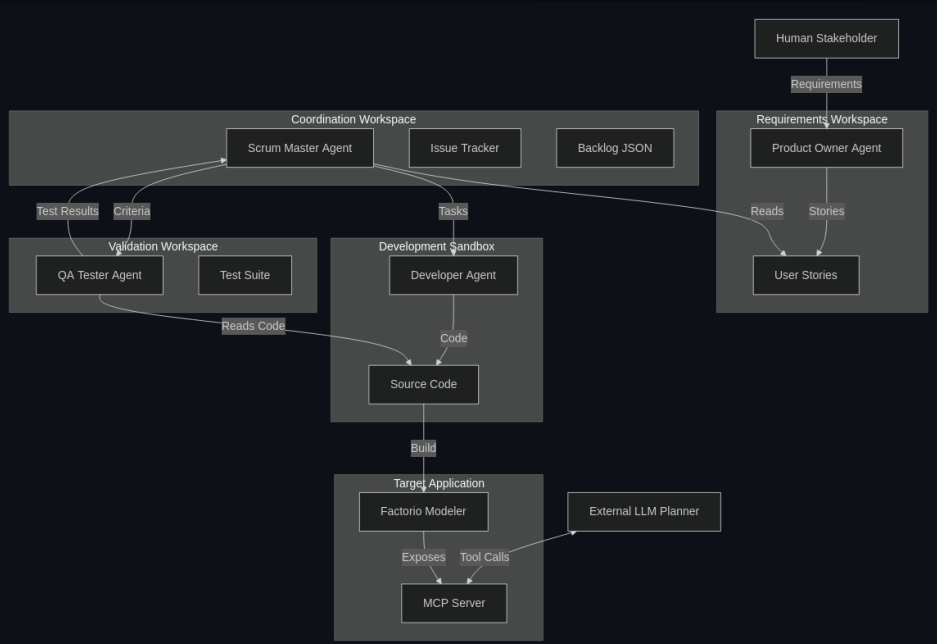


QA Tester

Tests & Validierung.
Stoppt fehlerhaften
Code.

Kein Agent darf Aufgaben anderer Rollen übernehmen (Boundary Enforcement).

SYSTEMARCHITEKTUR



Zentrale Merkmale

- **LangGraph Orchestrierung:** Deterministische Zustandsübergänge.
- **Isolierte Toolchains:** Jeder Agent hat nur Zugriff auf seine spezifischen Tools.
- **Git-Audit:** Jeder Fortschritt wird via Git-History dokumentiert.
- **Zustandsmanagement:** Zentrales Gedächtnis der Scrum-Session.

| TECHNOLOGIESTACK

Bereich	Technologie
Agent Framework	CrewAI
Workflow Engine	LangGraph
LLM Runtime	Ollama (Lokal)
Modelle	Omnocoder-9B / Qwen 3.5
Backend / GUI	.NET 8 / Avalonia UI
Protokoll	MCP (Model Context Protocol)

| LANGGRAPH STATE MACHINE

Der Entwicklungsprozess als deterministischer Workflow:



Logik: QA erfolgreich → Ende. QA fehlgeschlagen → Developer. Rollenkonflikt → Zurück zum Verursacher.

| SICHERHEITS- & TOOLING-KONZEPT



Sichere Dateioperationen

Eigene Wrapper (SafeFileWriter, SafeFileRead etc.) verhindern Directory Traversal und unbefugte Dateizugriffe.



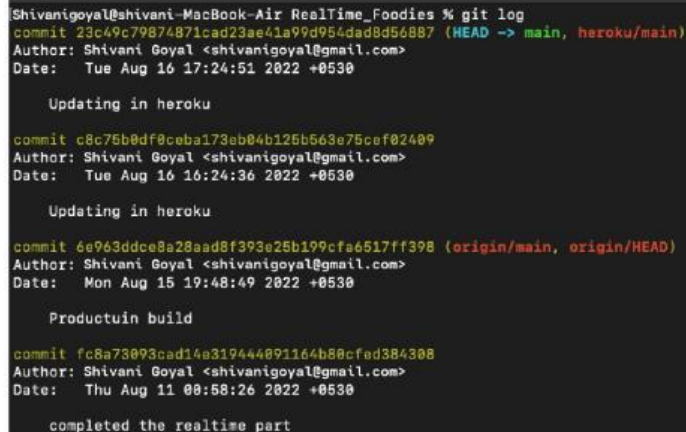
Path-Isolation

Jede Pfad-Anfrage wird gegen eine ALLOWED_BASE geprüft. Zugriff außerhalb des Workspaces ist technisch unmöglich.

| GIT-INTEGRATION & AUDIT

Die Agenten kommunizieren ihren Fortschritt über eine automatisierte Git-Toolchain:

- > Automatisches add und commit.
- > Erzwingung von **Conventional Commits** (feat, fix, docs).
- > Sperrung gefährlicher Operationen (push, remote).



```
Shivanigoyal@shivani-MacBook-Air RealTime_Foodies % git log
commit 23c49c79874871cad23ae41a99d954dad8d56887 (HEAD -> main, heroku/main)
Author: Shivani Goyal <shivanigoyal@gmail.com>
Date: Tue Aug 16 17:24:51 2022 +0530

    Updating in heroku

commit c8c75b0df0ceba173eb04b125b563e75cef02409
Author: Shivani Goyal <shivanigoyal@gmail.com>
Date: Tue Aug 16 16:24:36 2022 +0530

    Updating in heroku

commit 6e963ddce8a28aad8f393e25b199cfa6517ff398 (origin/main, origin/HEAD)
Author: Shivani Goyal <shivanigoyal@gmail.com>
Date: Mon Aug 15 19:48:49 2022 +0530

    Productuin build

commit fc8a73093cad14a319444091164b80cfed384308
Author: Shivani Goyal <shivanigoyal@gmail.com>
Date: Thu Aug 11 00:58:26 2022 +0530

    completed the realtime part
```

| DOCKER-SETUP & SANDBOX



Umsetzung

- > Zentraler Entwicklungs-Container.
- > Persistenter Workspace via **Volume Mounts**.
- > Schnittstelle zum lokalen Ollama Host.
- > Kontrollierte Laufzeitumgebung für .NET Builds.

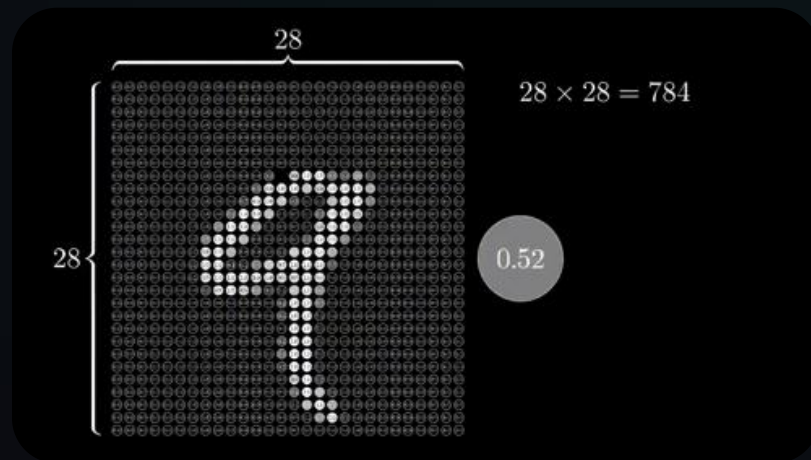
Sicherstellung von reproduzierbaren Builds unabhängig vom Host-System.

LOKALES LLM SETUP

Hauptmodell: Omnicoder-9B

Feingetuntes Qwen3.5 für coding agents.

- 16k Kontextfenster
- Niedrige Temperatur (0.2)
- Spezielle Stop Tokens (Observation:)
- ReAct Prompting Pattern



| REACT TOOL CALLING PATTERN

Das standardisierte Format für Agenten-Aktionen:

Thought: Ich muss die Projektstruktur lesen.

Action: SafeDirectoryRead

Action Input: {"directory_path": "."}

Herausforderung

Viele lokale Modelle neigten dazu, reines JSON zu generieren oder das ReAct-Format zu ignorieren, was zu Workflow-Abbrüchen führte.

| EVALUIERTE MODELLE IM VERGLEICH



Bewertung basierend auf Tool-Usage, Rollentreue und Codegüte.

PROBLEME MIT LLAMA 3.2 (3B)

- **Kontextverlust:** Vergisst vorangegangene Fehlversuche und wiederholt identische Kommandos.
- **Pfad-Verwirrung:** Ignoriert die vorgegebene Ordnerstruktur ab dem ersten Prompt.
- **Syntax-Fehler:** Generiert oft nicht-kompilierbaren C#-Code oder Fragmente.

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace LLama3_2_3B
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("LLama 3.2 (3B)");
            Console.WriteLine("Please enter a command:");
            string command = Console.ReadLine();
            while (command != "exit")
            {
                // Here you would call the LLaMA 3.2 (3B) model to process the command
                // For example: string response = Model.Process(command);
                Console.WriteLine(response);
                Console.WriteLine("Please enter a command:");
                command = Console.ReadLine();
            }
        }
    }
}
```

| PROBLEME MIT QWEN 2.5 CODER



Tool-Calling Defekt

Das Modell erzeugte Toolcalls oft als reines JSON-Objekt ohne den erforderlichen ReAct-Header. CrewAI konnte diese Aktionen nicht parsen.



Ignorierte Tools

Trotz expliziter Anweisungen wurden Git- und Dotnet-Tools kaum verwendet. Das Modell versuchte Aufgaben rein textuell zu lösen.

| ARCHITEKTURZERFALL (QWEN 3.5)

Eines der kritischsten Probleme war die mangelnde Konsistenz der Projektstruktur:

Casing & Duplikation

Modelle erstellten parallel:

- Models/
- models/
- Model/
- FactorioModeler/

Trotz expliziter Regeln (Singleton Architect) wurde der bestehende Workspace oft ignoriert. Dies führte zu einer unkontrollierbaren Fragmentierung der Codebasis.

| QWEN2.5:14B-INSTRUCT (Q4_K_M)

Hardware-Setup

GPU
RTX 4070 12GB

VRAM Auslastung
~45 Layer GPU

Rechenlast
Rest auf CPU

Quantisierung
Q4_K_M

Beobachtungen

- > Nur **~1/10 Toolcalls** funktionierte stabil.
- > Extreme Häufung von **Retry-Loops**.
- > Massiver **Kontextverlust** während der Session.
- > Duplicate Dateien & Casings ignoriert.



Auswirkungen & Maßnahmen

- > **10+ Iterationen** für minimale Änderungen.
- > Inakzeptabel lange Laufzeiten.
- > **Gegenmaßnahmen:**
 - > Neue Systemprompts
 - > ReAct-Format Anpassungen
 - > Modelfile-Tuning

ERGEBNIS: Keine signifikante Verbesserung

| TOOL CALLING FEHLER-ANALYSE

Typisches Fehlerbild

```
[NODE]: scrum_master  
  
[LOG]: SMA: Rejected - json  
{  
  "name": "safe_file_read",  
  "arguments": {  
    "file_path": "recipes.json"  
  }  
}
```

! Erkenntnis:

Lokale Modelle waren zu unzuverlässig für stabilen Agentic Tool Use.

Das Problem

- > Toolcall als **reiner Text/JSON** ausgegeben statt echte Ausführung.
- > Ignorieren der Prompt-Anweisung: *"Never output tool calls inside markdown code blocks"*.

Folgen

- > Kein Dateiinhalt geladen (fehlende Information).
- > Workflow wurde zurückgesetzt.
- > **Endlosschleifen:** Modell versucht denselben Fehler erneut.
- > Hohe Laufzeit ohne jeglichen Fortschritt.

| DAS QA-LOOP PROBLEM

Der Workflow geriet in eine "Todesschleife":



Lokale Modelle waren nicht fähig, aus den Build-Fehlermeldungen die korrekten Schlüsse für eine Reparatur zu ziehen.

| ITERATION LIMITS & KOSTENKONTROLLE

10

MAX_QA_DEV_ITERATIONS

- > Ursprünglich bis zu **2000 Iterationen** beobachtet.
- > Tausende Toolcalls führten zu CPU-Heißlauf ohne Ergebnis.
- > Hard-Exit nach 10 Zyklen zum Schutz der Hardware und Zeit.

TATSÄCHLICH ERZEUGTE COMMITS

commit b19e42c824dd382c22928a305ec72d51afe1a566

Author: Scrum Master Bot

Date: Thu May 28 06:16:04 2026

feat: update sub-system READMEs and specs

docs: Update Factorio Modeler documentation to rev 4.0.0

Auffällig: Dokumentation und Prozess-Updates funktionierten deutlich zuverlässiger als die tatsächliche C#-Implementierung.

| POSITIVE ERGEBNISSE & ERFOLGE



Architektur

Hervorragende READMEs und strukturierte Modulbeschreibungen.



Prozesssimulation

Das Rollenmodell und Routing via LangGraph waren stabil.



Spezifikation

PO-Agent erzeugte valide Gherkin Stories und Akzeptanzkriterien.

NEGATIVE ERGEBNISSE & LIMITS

- **Codegenerierung:** Meist nur Skelette ohne Logik oder syntaktisch fehlerhaft.
- **Tool Calling:** Konstante Brüche im ReAct-Format bei kleinen Modellen.
- **Self-Correction:** Fast keine Fähigkeit zur autonomen Fehlerbehebung.

```
def __init__(self, model_name="gpt-4o", max_tokens=1000, temperature=0.7):
    self.model_name = model_name
    self.max_tokens = max_tokens
    self.temperature = temperature

def generate(self, prompt):
    """Generate text based on the prompt using the specified model and parameters.
    The prompt should be a string, and the response will be a string of generated text.
    The response length will be limited by max_tokens.
    """
    # Construct the request payload
    payload = {
        "model": self.model_name,
        "prompt": prompt,
        "max_tokens": self.max_tokens,
        "temperature": self.temperature,
    }

    # Send the request to the API
    response = self._send_request(payload)

    # Extract the generated text from the response
    generated_text = response.get("text", "")

    return generated_text

def _send_request(self, payload):
    """Send a request to the API and return the response.
    This is a placeholder for the actual API call logic.
    """
    # Simulate a network call
    import random
    import time

    # Simulate a delay
    time.sleep(1)

    # Simulate a response
    response = {
        "text": f"Generated text based on the prompt: {payload['prompt']}.",
    }

    return response
```

//

Das Problem war nicht die Architektur oder die Software-Wahl, sondern die Leistungsfähigkeit lokaler Modelle.

MAGE-Scrum Team-Audit

| ABSCHLIESSENDES FAZIT

Erfolgreich

- > Multi-Agent Architektur
- > Workflow-Orchestrierung
- > Tooling & Sandbox-Konzept
- > Dokumentationsgenerierung

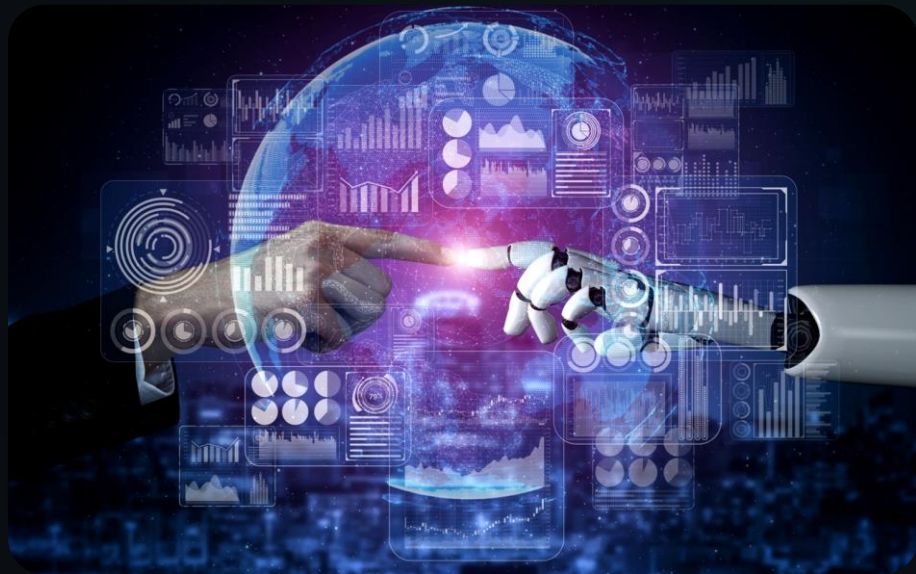
Nicht erfolgreich

- > Vollständige Softwareentwicklung
- > Stabile Codegenerierung
- > Automatisierte QA-Zyklen
- > Lauffähiger Factorio Modeler

Lokale Small-LLMs (bis 9B) sind aktuell unzureichend für autonome Coding-Agents.

| FUTURE WORK & AUSBLICK

- Einsatz von **70B+ Modellen** (Llama 3 / Qwen) für bessere Planung.
- **AST-basierte** Codevalidierung vor dem Build.
- Implementierung eines **LangLong-Memory** Systems.
- Echte Container-Isolation pro Agent.



Demo / Live Walkthrough



Einblick in die generierte Struktur.

Vielen Dank!

Haben Sie noch Fragen zu MAGE-Scrum?

Fabian Rahofer | Hannes Zelger